

Code-Layout, Readability and Reusability

Date	
Team ID	xxxxxx
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the EduGenie codebase in terms of its structure, readability and reusability.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation (PEP 8) in Python; 4 spaces in HTML/CSS/JS
2	Proper File Structure	Files and folders are logically organized	Yes	One module per feature; templates/, static/, tests/ folders; config in .env
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. num_questions, explain_backend, MAX_TEXT_CHARS
4	Function / Method Names	Functions are descriptively named	Yes	answer_question_with_gemini, generate_quiz, clean_json_block, get_learning_recommendations
5	Code Comments	Inline and block comments explain logic	Yes	Module docstrings, function docstrings and comments for non-obvious logic
6	Modular Design	Code is split into reusable functions/modules	Yes	Shared gemini_client.generate() used by all features; prompts defined as constants
7	No Redundant Code	Duplicate or unused code is removed	Yes	Common validation and error helpers in main.py; pyflakes reports no issues
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Custom GeminiError hierarchy mapped to HTTP 400/502/503; local model falls back to Gemini

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	gemini_client.py (generate(), GeminiError)	Python, google-genai SDK	Q&A, explanation, quiz, summary, learning-path modules	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
2	quiz_module.clean_json_block() / parse_quiz()	Python (json, re)	Quiz generation; reusable for any LLM JSON output	High
3	validate_input() / ai_error() helpers (main.py)	Python, FastAPI	All five API endpoints	High
4	renderMarkdown() / callApi() (script.js)	JavaScript	Rendering every AI result and every API call in the UI	High
5	tests/conftest.py fake_gemini fixture	Python, pytest	All API and module tests (no real API calls needed)	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of frontend, API layer, feature modules and AI gateway
Readability	5	Descriptive names, type hints, short functions, docstrings
Reusability	4	Shared Gemini client and helpers; prompts could later move to a config file
Documentation / Comments	5	README with setup/run/test steps, module docstrings, .env.example, API docs at /docs
Overall Score	4.75 / 5	Clean, modular and tested (40 automated tests passing)